Web 信息处理与应用

实验一　信息检索

Qilin-IR-50K 中文信息检索实验报告

组长	龚夏瑜
小组成员	龚夏瑜　PB24331921
	黄蕴　PB24331918
	陈熙　PB24331924

运行环境	Windows 11；Intel i7-13650HX；16 GB 内存；Python 3.13
附件索引	https://pan.ustc.edu.cn/group/home/60002193
报告日期	2026 年 10 月

摘　要

本实验以 Qilin-IR-50K 的 50,000 篇中文笔记为对象，比较分词方法，构建文档级和位置级倒排索引，并测试索引压缩、布尔检索、TF-IDF 与语义检索。FMM 保留停用词后得到 115,215 个词项和 4,074,930 条词项–文档关系。Front Coding 与 Gap + VByte 组合将二进制索引由 79.66 MB 压缩至 25.21 MB，节省 68.35% 的空间；暖缓存条件下，布尔检索平均耗时增加约 18%。在倒排表长度差异较大的一条查询中，平方根步长跳表使比较次数减少 89.1%，全量指针存储约占 Raw 倒排记录的 2.8%。基于 12 条查询、333 条人工相关性标注，原文语义编码的平均 P@10 和 MRR@10 分别为 0.533 和 1.000，均高于本实验中的 TF-IDF 与词项向量聚合方法。上述质量比较受查询规模、标注范围和编码输入差异的限制。

1 实验设置

1.1 数据与任务

课程实验包括分词与规范化（任务 1）、倒排表构建（任务 2）、位置索引与压缩（任务 3），以及布尔检索、向量空间模型和预训练模型检索（任务 4）[**?**]。实验使用教学子集 Qilin-IR-50K 的 `documents.jsonl`，每篇文档包含原始编号 `doc_id`、标题 `title` 和正文 `content`。检索质量评价采用 12 条查询，其中 11 条为自拟查询，1 条取自 `queries.jsonl`；数据集查询日志未用于定量评价。

程序使用 Python、jieba、NumPy、SciPy、Sentence Transformers 和 PyTorch，模型均在 CPU 上运行。各阶段中间结果保存为 JSONL、二进制索引、CSR 稀疏矩阵或 NumPy 数组。

1.2 实验流程与统计口径

先在 100 篇和 1,000 篇文档上进行正确性检查，再对 50,000 篇文档运行完整实验。本文表格和结论均采用全量实验数据。其余统计约定如下：

1. 文件大小采用十进制单位，即 $1\,\mathrm{MB} = 10^6$ 字节。
2. 检索计时在词典预先加载、缓存已预热的条件下进行，以多次运行的中位数作为单条查询耗时；布尔检索每组重复 10 次。
3. 不同模型的余弦相似度数值不直接比较，检索质量统一以相关性评价指标衡量。
4. 代码文件与输出数据的对应关系见附录 A。

2 文档分词与规范化（任务 1）

2.1 分词方法

标题和正文分别分词，并分别记录位置；任务 3 的短语匹配不跨字段。统计词项数量时，合并两个字段的分词结果。实验采用以下两种方法：

- **前向最大匹配（FMM）**：使用 jieba 内置词典的词频表，最大词长设为 6。中文片段按最长词匹配，英文、数字和符号等非中文片段按字符类别切分。
- **隐马尔可夫模型（HMM）**：调用 jieba 的 `finalseg` 模块，使用转移概率和发射概率进行 Viterbi 解码，不启用 jieba 的混合分词模式。

2.2 规范化处理

两种分词结果采用相同的规范化步骤：进行 NFKC 规范化，将英文转为小写，将纯数字词项（含小数）替换为 `<NUM>`，并去除标点及其他特殊符号。随后根据自定义的 31 词停用词表，分别生成保留停用词（Keep）与去除停用词（Drop）的词项序列，用于任务 4 的对照实验。

2.3 结果对比与分析

表 1 50,000 篇文档的分词统计

指标	FMM	HMM
分词耗时 / s	15.411	39.471
Keep 词项出现次数	6,759,956	6,631,225
Drop 词项出现次数	5,999,688	5,750,165
Keep 词汇表大小	115,215	388,632
Drop 词汇表大小	115,184	388,601
停用词移除比例	11.25%	13.29%

数据来源：`output/1/segmentation_stats_all.json`。

本次测试中，HMM 的分词耗时约为 FMM 的 2.56 倍，Keep 词汇表规模约为 FMM 的 3.37 倍。以下样例展示了两种方法的切分差异：

- 标题“C 哥预测还准不准”中，FMM 将相关片段切为“准 / 不准”，HMM 则保留为“准不准”。
- 标题“手持挂烫机奥克斯”中，HMM 将“机奥克斯”合并为一个词项。FMM 分开了“机”和品牌名，但也将“挂烫”切为“挂 / 烫”。

这些样例说明，两种方法均存在边界处理不当的情况。HMM 较大的词汇表可能与不同的切分边界有关，但仅凭词汇表规模和少量样例，不能判断整体分词质量。后续倒排索引、压缩和布尔检索采用 FMM-Keep；TF-IDF 与语义检索另设 Keep / Drop 对照。

3 倒排表构建（任务 2）

3.1 文档级倒排索引

按文档输入顺序分配从 0 开始的内部编号 d，并保留其与原始 `doc_id` 的映射。每个词项 t 对应一个升序文档编号列表，同一文档只记录一次；列表长度即文档频率 $df(t)$。构建时先使用 `array('I')` 保存倒排表，再按词项排序输出 JSONL。

3.2 单层跳表指针

为减少交集运算中的逐项比较，在倒排表上等距设置单层跳跃指针。对于长度为 $df(t)$ 的倒排表，默认步长为

$$s(t) = \max\left(2, \lfloor \sqrt{df(t)} \rfloor\right), \tag{1}$$

在位置 $0, s, 2s, \ldots$ 处设置指向下一跳点的指针，$df < 4$ 的短表不设置跳跃边。该规则在全量索引中生成 263,858 条跳跃边；不同步长的存储和检索开销见第 5.1.4 节。

表 2 文档级倒排索引统计

指标	结果
索引文档数 / 词汇量	50,000 / 115,215
词项–文档关系数（$\sum_t df(t)$）	4,074,930
单层跳跃边总数（$\sqrt{df}$ 规则）	263,858
最高文档频率词项	“话题”（$df = 46{,}024$）
文档级 JSONL 大小 / 构建耗时	33.62 MB / 3.43 s

数据来源：`output/2/index_stats_fmm_keep_all.json`。

使用 `02_probe_index.py` 检查索引结构。全部 115,215 个词项的倒排表均严格升序且无重复，跳跃边未越界，实际步长与设定一致。本实验未实现课程选做的多层跳表，以下分析仅涉及单层等距跳表。

4 倒排表扩展与压缩（任务 3）

4.1 词项位置索引

将倒排记录扩展为三元组 $(d, P_t^{\text{title}}, P_t^{\text{body}})$。标题和正文分别从 0 开始记录规范化、保留停用词后的词项下标，以保留停用词所在的位置；短语匹配限定在同一字段内。位置索引采用分桶和外部排序构建，使用 64 个临时分桶，每条中间记录占 10 字节。全量数据包含 6,759,956 个词项位置，构建耗时为 19.15 s。分桶处理可减少一次性载入的记录数。

表 3 位置索引统计

指标	结果
词项位置出现次数	6,759,956（与 FMM-Keep 词项数完全一致）
词项–文档关系数	4,074,930（与任务 2 完全一致）
位置索引 JSONL 大小 / 构建耗时	82.30 MB / 19.15 s

数据来源：`output/3/positional_stats_fmm_keep_all.json`。

使用 `03_check_positions.py` 逐词核对后，位置索引中的文档编号列表与任务 2 的倒排表一致，各字段的位置列表也均严格升序。

4.2 索引压缩：Front Coding 与 Gap + VByte

按照课程课件介绍的方法 [**?**]，分别压缩词典和倒排记录。四组对照使用相同的位置索引，仅改变编码方式：

- **Front Coding**：将排序后的词典按每块 8 个词项分组，块首保存完整词项，其余词项保存与前一词项的公共前缀字节数及后缀。
- **Gap + VByte**：文档编号和位置均保存相邻差值，首项相对 -1 取差，使差值为正；随后采用含 7 位数据位和 1 位终止标志的变长字节编码。Raw 对照组使用 u32 定长编码。

表 4 四种索引编码的空间占用

方案	词典 / MB	倒排 / MB	元数据 / MB	总计 / MB	节省率
Raw	0.956	75.939	2.765	79.660	—
Front	0.614	75.939	2.765	79.318	0.43%
VByte	0.956	21.830	2.765	25.551	67.93%
Combined	0.614	21.830	2.765	25.209	68.35%

数据来源：`output/3/compression_summary_fmm_keep_all.json`。

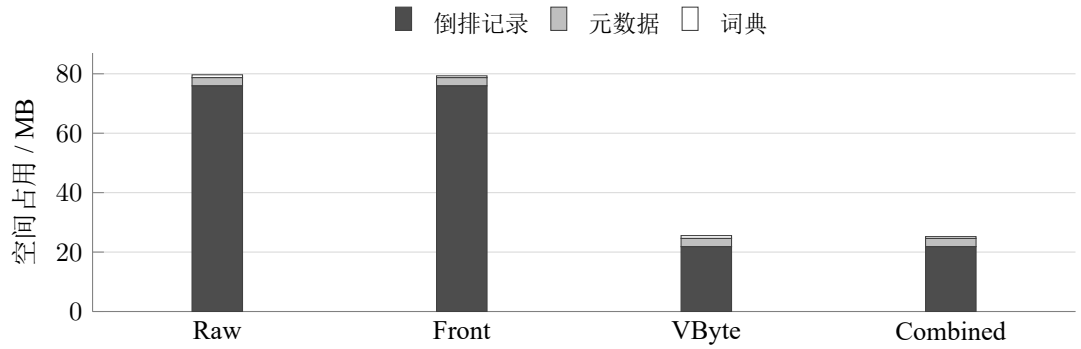


图 1 四种方案的二进制索引空间构成

Front Coding 使词典体积减少 35.78%。由于词典仅占原索引的约 1.2%，总空间减少 0.43%。Gap + VByte 将倒排记录从 75.94 MB 压缩至 21.83 MB，减少 71.25%，是组合方案节省空间的主要来源。

上述比例均以字段相同的二进制 Raw 索引（79,659,905 B）为基准，不包含 JSONL 转为二进制格式带来的体积变化。总大小包括词典、倒排记录和索引元数据，不包括共享文档映射。通过 `03_verify_compression.py` 校验，Combined 模式解压后的 115,215 个词项、4,074,930 条倒排记录和 6,759,956 个位置均与原索引一致；其余模式各抽样 100 个词项，结果一致。

5 信息检索方法比较（任务 4）

5.1 布尔检索（任务 4A）

5.1.1 复杂布尔表达式与处理顺序

设计 9 条布尔查询，覆盖 AND、OR、NOT、嵌套括号和纯 NOT 表达式，所用词项均在 FMM 词汇表中。基准测试采用 Raw 索引，按表达式的原始顺序处理，并关闭跳表。

表 5 9 条布尔查询的检索结果

查询表达式	命中数	中位耗时 / ms	比较次数
((登机牌 AND 习惯) OR 收藏) AND NOT 话题	46	134.77	5,409
(平价 OR 护肤品) AND (学生 AND 党)	157	4.76	4,052
(山西 AND 壁画) OR (古建 AND 艺术)	2	1.34	787
(旅行 AND 攻略) AND NOT 广告	1,016	19.22	54,253
(考研 AND (英语 OR 政治)) AND 经验	20	5.56	4,076
(北京 AND 故宫) OR (西安 AND 兵马俑)	26	1.64	996
(咖啡 AND (做法 OR 教程)) AND NOT 广告	19	14.37	51,579
NOT (话题 OR 分享)	3,749	177.63	46,240
(分享 AND 攻略) AND 旅行（高频词优先）	182	22.16	14,084

数据来源：`output/4/boolean_benchmark_fmm_all.json`。

处理顺序。分别按原始左结合顺序和基于 df 的估计顺序执行查询；后者优先处理 AND 链中预计结果较小的子表达式。本次实验中，重排前后的比较次数基本一致。例如，高频词优先的查询分别需要 14,084 和 14,096 次比较，差异小于 0.1%，未观察到稳定的耗时改善。

线性双指针归并的单次交集最坏时间复杂度为 $O(|A|+|B|)$。执行顺序仍会影响中间结果的长度，但本次查询集未体现出明显收益，不能据此认定重排对所有查询均无效。含高频词“话题”（$df = 46{,}024$）和纯 NOT 补集运算的查询耗时较高，倒排解码及补集处理可能是主要开销。

5.1.2 压缩前后检索效率

四种编码方式使用相同的 9 条查询，均保持原始处理顺序并关闭跳表。下表先以每条查询的重复运行中位数作为单条耗时，再计算 9 条查询间的平均值和中位数。

表 6 四种索引编码的布尔检索效率（9 条查询汇总）

方案	平均耗时 / ms	中位耗时 / ms	词典加载 / s	索引大小 / MB
Raw	42.38	14.37	0.164	79.66
Front	42.70	14.23	0.195	79.32
VByte	49.49	15.04	0.186	25.55
Combined	50.07	15.00	0.196	25.21

Combined 的空间节省率为 68.35%，暖缓存条件下的平均耗时由 42.38 ms 增至 50.07 ms，增加约 18%。这表明在本实验实现中，压缩并未缩短检索时间，可能与逐字节解码的 CPU 开销有关。Front 与 Raw 的查询耗时接近。冷缓存、部分倒排解码等条件下的表现尚未测试，不能由当前结果推断。

5.1.3 短语检索与位置索引的效果

对 5 个短语进行位置检索，要求词项在同一字段内按顺序相邻，并与相同词项的普通 AND 查询比较。过滤率定义为 1 − 短语命中数/AND 命中数。

表 7 短语检索与普通 AND 查询的结果对照

短语	短语命中	AND 命中	过滤率	Raw / ms	Combined / ms
收藏登机牌	1	1	—	1.24	1.41
平价护肤品	2	13	84.6%	1.67	1.84
北京故宫	2	23	91.3%	1.41	1.74
减肥运动	4	187	97.9%	3.71	4.41
旅行攻略	264	1,018	74.1%	9.62	11.40

以“减肥运动”为例，普通 AND 查询返回 187 篇包含两个词项的文档，短语检索保留其中 4 篇，过滤率为 97.9%。两者的差异来自是否要求词项在同一字段内相邻。“收藏登机牌”在两种方式下均命中 1 篇，说明这条查询未产生额外过滤。Combined 的短语检索耗时均高于 Raw；按表中数据计算，增幅约为 10–23%。

5.1.4 跳表步长对存储与检索的影响

选择以下 3 条 AND 查询，比较无跳表及 5 种跳表步长。每条跳跃边保存目标文档号和倒排内偏移，均为 u32，因此按每条 8 B 估算指针存储开销。

- Q1：（话题 AND 登机牌）`AND` 习惯；
- Q2：（攻略 AND 旅行）`AND` 民宿；
- Q3：（考研 AND 英语）`AND` 复习。

表 8 不同跳表步长下的比较次数与中位耗时（Raw 索引）

查询	无跳表	2	4	8	16	$\sqrt{df}$
Q1 比较次数	46,189	23,194	11,717	6,083	3,445	5,018
中位耗时 / ms	160.2	139.2	134.9	131.8	131.0	136.5
Q2 比较次数	5,707	6,838	6,324	6,145	5,983	5,810
中位耗时 / ms	8.97	9.26	9.33	9.39	9.30	9.27
Q3 比较次数	1,576	1,875	1,785	1,738	1,664	1,648
中位耗时 / ms	2.77	2.83	2.88	2.88	2.75	2.85

表 9 不同跳表步长的指针存储开销

步长	跳跃边总数	指针存储 / MB	占 Raw 倒排记录比例
2	1,949,530	15.60	20.5%
4	962,007	7.70	10.1%
8	467,852	3.74	4.9%
16	224,561	1.80	2.4%
$\sqrt{df}$	263,858	2.11	2.8%

数据来源：`output/4/skip_storage_fmm_all.json`。

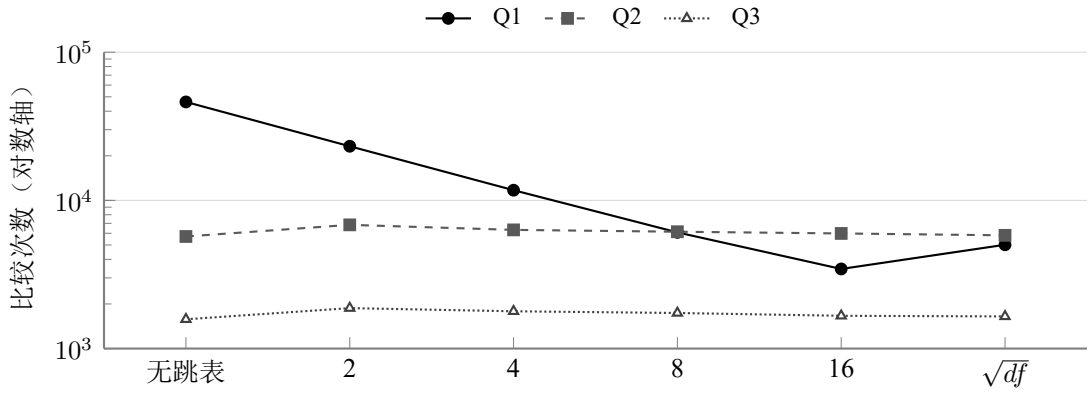


图 2 三条查询的比较次数随跳表步长变化（对数轴）

Q1 中，“话题”的文档频率为 46,024，“登机牌”为 21，倒排表长度差异较大。平方根步长使比较次数减少 89.1%，中位耗时由 160.2 ms 降至 136.5 ms，减少约 14.8%；固定步长 16 的比较次数减少 92.5%，耗时降至 131.0 ms。时间降幅小于比较次数降幅，与跳表未减少倒排解码量有关。

Q2 和 Q3 中，跳表未带来稳定的耗时改善。固定步长 2 反而使比较次数分别增加约 19.8% 和 19.0%，因为判断能否跳跃也需要比较。平方根步长的比较次数增幅分别约为 1.8% 和 4.6%。

平方根步长的指针估算大小为 2.11 MB，占 Raw 倒排记录的 2.8%，低于固定步长 2 的 15.60 MB。在这 3 条查询上，它能减少 Q1 的比较次数，对 Q2、Q3 的影响较小；固定步长 16 则使用更少的指针，并在 Q1 上取得更低耗时。因此，当前结果不足以确定普遍最优的步长。

5.2 向量空间模型（任务 4B）

采用原始词频与平滑 IDF 计算权重，文档和查询使用相同规则，标题与正文同权。文档矩阵采用 `float32` 类型的 CSR 稀疏存储，相关性由余弦相似度计算：

$$w_{t,d} = tf(t,d)\left[\ln\frac{N+1}{df(t)+1}+1\right], \qquad \mathrm{sim}(q,d)=\frac{\mathbf{q}^{\mathsf{T}}\mathbf{d}}{\|\mathbf{q}\|_2\|\mathbf{d}\|_2}. \tag{2}$$

表 10 TF-IDF 模型的构建统计

指标	Keep	Drop	下降比例
词汇表规模	115,215	115,184	0.027%
词项出现次数	6,759,956	5,999,688	11.25%
矩阵非零元素	4,074,930	3,800,308	6.74%
CSR 文件 / MB	22.73	21.40	5.85%
构建耗时 / s	9.41	8.56	9.06%

删除 31 个停用词后，词项出现次数减少约 76 万。使用与语义检索相同的 12 条查询比较 Keep 和 Drop 模型；候选数指与查询具有正相似度的文档数，Jaccard@10 为两个 Top-10 文档集合的交集大小与并集大小之比。

表 11 TF-IDF 保留与去除停用词的检索结果

查询	Keep 候选	Drop 候选	Top-10 重合	Jaccard@10
收藏登机牌	821	821	10/10	1.000
有收藏登机牌的习惯吗	42,849	18,817	10/10	1.000
山西古建壁画	86	86	10/10	1.000
平价护肤品学生党	2,518	2,518	9/10	0.818
旅行攻略民宿	4,555	4,555	10/10	1.000
考研英语复习经验	2,887	2,887	10/10	1.000
咖啡做法教程	2,476	2,476	9/10	0.818
北京故宫门票	1,463	1,463	10/10	1.000
减肥运动食谱	2,179	2,179	10/10	1.000
衣柜颜色搭配	3,196	3,196	10/10	1.000
酒酿好处（真实查询）	339	339	10/10	1.000
适合学生党的平价护肤品…（长查询）	43,048	23,351	10/10	1.000

数据来源：`output/4/tfidf_comparison/`。

12 条查询中，有 10 条的 Keep / Drop Top-10 集合相同，另 2 条各有 9 篇重合。对含功能词的第 2、12 条查询，去除停用词后，候选数分别从 42,849 降至 18,817、从 43,048 降至 23,351，但 Top-10 集合未变。本次结果表明，停用词处理对这两条查询的候选规模影响较大，对高排名集合的影响较小；集合相同并不表示内部排序完全一致。

5.3 语义检索（任务 4C）

5.3.1 方法与规模

使用 `BAAI/bge-small-zh-v1.5` 生成 512 维向量，最大输入长度为 512 个模型 token，在 CPU 上推理。Raw 直接编码“标题 + 正文”原文；Seg-Keep 对 FMM 词项逐词独立编码，再按词频加权平均并进行 L2 归一化；Seg-Drop 在聚合前去除停用词。三种方法均使用模型自带的 tokenizer，另设 TF-IDF 作为词汇匹配基线。

表 12 50,000 篇文档的语义向量规模与记录耗时

方法	文档数	维度	向量文件 / MB	记录耗时 / s
Raw	50,000	512	102.40	1,277.9
Seg-Keep	50,000	512	102.40	8.5*
Seg-Drop	50,000	512	102.40	10.5*

* Seg 计时对应词项向量缓存（115,215 词独立编码）建立后的文档聚合与写盘，未计入首次词项编码成本；不可与 Raw 的完整推理时间直接比较。

Raw 与 Seg 的差异不只涉及预先分词：Raw 的输入超过 512 个模型 token 时会被截断，而 Seg 聚合使用全部规范化词项；Seg 还替换了数字并删除标点。此外，逐词编码后平均会改变文本的上下文表示。因此，这一对照不能单独用于评价预先分词的影响。

5.3.2 分词与停用词对结果集合的影响

表 13 语义检索方法的 Top-10 集合重合程度（Jaccard@10）

查询	Raw / Seg-Keep	Seg-Keep / Seg-Drop
收藏登机牌	0.111	0.429
有收藏登机牌的习惯吗	0.000	0.000
山西古建壁画	0.053	0.538
平价护肤品学生党	0.053	0.667
旅行攻略民宿	0.053	0.667
考研英语复习经验	0.053	0.538
咖啡做法教程	0.333	0.538
北京故宫门票	0.111	1.000
减肥运动食谱	0.000	0.818
衣柜颜色搭配	0.111	0.538
酒酿好处	0.053	0.333
适合学生党的平价护肤品…（长查询）	0.053	0.333
平均	0.082	0.492

数据来源：`output/4/semantic/comparison_all_*.json`。

Raw 与 Seg-Keep 的平均 Jaccard@10 为 0.082，说明两种方法返回的 Top-10 集合差异较大。逐词独立编码后求平均不保留词序，也不直接表示上下文中的词项搭配，这可能是差异的原因之一；输入截断和规范化差异也可能产生影响。Seg-Keep 与 Seg-Drop 的平均 Jaccard@10 为 0.492，低于 TF-IDF Keep / Drop 的集合重合程度，说明本次测试中停用词对词项向量聚合结果的影响更大。

5.3.3 人工相关性评价（P@10 / MRR@10）

将 12 条查询、4 种方法的 Top-10 结果按“查询–文档”对去重，共得到 333 条标注，其中相关 98 条、不相关 235 条。标注模板由脚本生成，再根据标题和正文预览逐条判断相关性，标签取 0 或 1。评价指标为：

$$P@10(q) = \frac{\#\{\text{前 10 篇中标为相关的文档}\}}{10}, \qquad \text{MRR}@10 = \frac{1}{|Q|}\sum_{q\in Q}\frac{1}{r_q}, \tag{3}$$

其中 r_q 为查询 q 首个相关文档的排名，前 10 篇无相关结果时令 $1/r_q = 0$。

表 14 各查询的 P@10 与汇总指标

查询	Raw	Seg-Keep	Seg-Drop	TF-IDF
收藏登机牌	0.10	0.10	0.10	0.10
有收藏登机牌的习惯吗	0.10	0.00	0.10	0.10
山西古建壁画	0.20	0.10	0.20	0.20
平价护肤品学生党	0.70	0.10	0.10	0.00
旅行攻略民宿	1.00	0.90	0.80	0.50
考研英语复习经验	0.80	0.40	0.30	0.40
咖啡做法教程	0.80	0.80	0.70	0.60
北京故宫门票	0.30	0.00	0.00	0.10
减肥运动食谱	1.00	0.90	0.90	0.80
衣柜颜色搭配	0.60	0.20	0.20	0.10
酒酿好处	0.40	0.10	0.10	0.40
适合学生党的平价护肤品…（长查询）	0.40	0.00	0.10	0.00
平均 P@10	0.533	0.300	0.300	0.275
MRR@10	**1.000**	0.561	0.771	0.750

数据来源：`output/4/semantic/precision_all_*.json`。

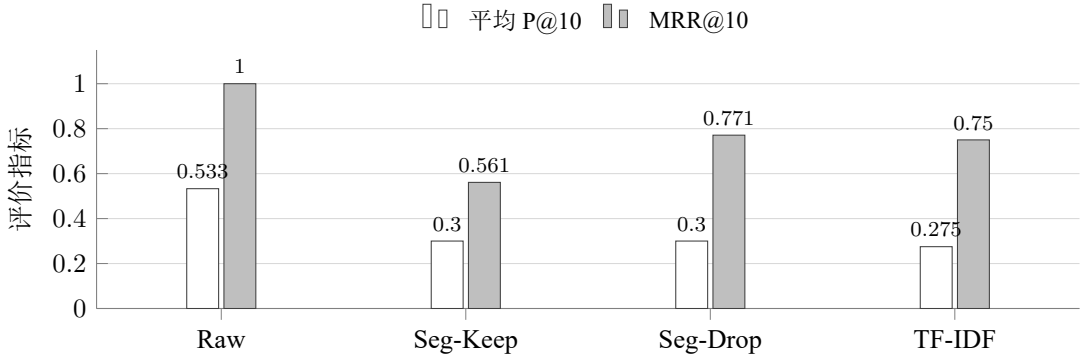


图 3 四种检索方法的平均 P@10 与 MRR@10（12 条查询）

Raw 在本次查询集上的平均 P@10 为 0.533，MRR@10 为 1.000，即每条查询的首位结果均被标为相关。这两项汇总指标高于另外三种方法。Seg-Keep 与 Seg-Drop 的平均 P@10 均为 0.300，MRR@10 则分别为 0.561 和 0.771，表明去除停用词后首个相关结果的平均排名有所改善，但前 10 篇的平均相关比例未提高。TF-IDF 在“减肥运动食谱”和“咖啡做法教程”上的 P@10 分别为 0.80 和 0.60，在“平价护肤品学生党”上为 0。

5.3.4 案例分析

- **“平价护肤品学生党”**：Raw 的 Top-10 中有 7 篇相关，Seg-Keep 和 Seg-Drop 各有 1 篇，TF-IDF 为 0 篇。该查询同时涉及价格、品类和适用人群，仅匹配部分词项的文档未必满足完整需求。
- **“北京故宫门票”**：Raw 的 P@10 为 0.30，检出的《逛故宫，去之前的准备工作》包含预约与门票信息；Seg-Keep 的 P@10 为 0。这一结果与逐词聚合难以保留上下文关联的特点相符，但不能据此排除其他输入差异的影响。
- **“减肥运动食谱”与“咖啡做法教程”**：四种方法的 P@10 均在 0.60–1.00 之间。查询词能较直接地描述文档主题，TF-IDF 在这两个案例中也能获得较多相关结果。

- **“酒酿好处”**：该查询取自数据集。Raw 与 TF-IDF 的 P@10 均为 0.40，Seg-Keep 与 Seg-Drop 均为 0.10。单个案例尚不足以说明造成差异的具体原因。

6 讨论与结论

6.1 主要结论

1. **分词。**本次运行中，HMM 耗时约为 FMM 的 2.56 倍，Keep 词汇表规模约为 FMM 的 3.37 倍。两种方法均存在切分不当的样例，尚未进行系统的分词准确率评价。
2. **索引与压缩。**文档级索引和位置索引包含相同的 4,074,930 条词项–文档关系。Combined 模式经全量解压校验，空间节省率为 68.35%，其中倒排记录的 Gap + VByte 编码贡献了主要压缩量。
3. **布尔检索。**AND 链重排在本次查询集上未带来明显收益。倒排表长度差异较大的 Q1 中，平方根步长跳表使比较次数减少 89.1%，耗时减少约 14.8%；另外两条查询未表现出稳定的耗时改善。Combined 在暖缓存条件下的平均检索耗时较 Raw 增加约 18%。
4. **短语检索。**位置条件可排除不满足字段内连续匹配的文档。5 条查询中，1 条与普通 AND 的结果相同，其余 4 条的过滤率为 74.1–97.9%。
5. **检索质量。**基于 12 条查询和 333 条标注，Raw 的平均 P@10 与 MRR@10 分别为 0.533 和 1.000，高于另外三种方法。停用词处理对 TF-IDF 的 Top-10 集合影响较小，对词项聚合结果及 MRR@10 的影响较大。

6.2 限制

本实验有以下限制：

1. 查询仅有 12 条，且部分主题相近，结果不能代表更广泛的检索需求。
2. 标注池仅覆盖四种方法的 Top-10 并集，无法估计全集召回率。相关性由一人根据标题和正文预览判断，未进行标注者间一致性检验。
3. Raw 与 Seg 在输入截断、规范化和聚合方式上存在差异，不能将质量变化完全归因于分词；Seg 计时还未包含首次词项编码成本。
4. 检索计时采用暖缓存条件，尚未测试冷缓存和按需解码，也未对更大规模查询集验证跳表步长的效果。

后续可扩大真实查询样本，并将 `search_logs` 中的点击记录作为补充相关性信号，同时保留人工判断，以进一步检查结论的适用范围。

A 代码与输出数据对照

全部脚本在项目根目录执行，命令形式为 `python src/脚本名.py`。100 篇和 1,000 篇规模用于正确性检查；`all` 表示 50,000 篇全量数据。各任务的代码与主要输出如下，具体命令见 `src/README.md`。

表 15 实验任务、代码文件与输出数据对照

任务	代码文件	主要输出（相对 output/）
数据体检	`00_check_data.py`	`data_check.json`
任务 1	`01_segment.py`、`01_compare_samples.py`	`1/tokens_*.jsonl`、`1/segmentation_*`
任务 2	`02_build_index.py`、`02_probe_index.py`	`2/inverted_*.jsonl`、`2/index_stats_*.json`
任务 3	`03_build_positions.py`、`03_compress_index.py`、`stage3b_codec.py`、`03_check_positions.py`、`03_verify_compression.py`	`3/positional_*.jsonl`、`3/compressed_*/`、`3/compression_*`
任务 4A	`04A_boolean_retrieval.py`、`04A_skip_storage.py`	`4/boolean_benchmark_*.json`、`4/skip_storage_*.json`
任务 4B	`04B_build_tfidf.py`、`04B_search_tfidf*.py`	`4/tfidf_fmm_*.npz` 等、`4/tfidf_comparison/`
任务 4C	`04C_build/compare/evaluate_semantic.py`	`4/semantic/`（向量、对照、标注 CSV、P@10）